



A Universal Framework for Vehicle Routing Problems with Large Language Models

Rui-Bin Zeng¹ · Mo-Xuan Yang² · Ming-Long Lei³ · Ling-Feng Niu²  · Yu-Hong Dai⁴

Received: 28 May 2025 / Revised: 11 March 2026 / Accepted: 16 March 2026

© Operations Research Society of China, Periodicals Agency of Shanghai University, Science Press, and Springer-Verlag GmbH Germany, part of Springer Nature 2026

Abstract

Vehicle routing problems (VRP) are typical NP-hard combinatorial optimization problems that have great significance in real-world applications. Recently, neural methods have shown promising abilities to solve VRP with superior quality while maintaining high efficiency. Existing neural methods still have limitations in terms of data requirements, especially for VRP variants. Furthermore, it is difficult to generalize across different types of VRPs with a unified learning framework without extensive retraining. Notably, the large language models (LLMs) have shown potential to assist in solving VRP problems. Therefore, this paper proposes a universal framework for VRP based on LLMs. This framework captures the essential characteristics of different VRPs in a unified manner and adapts to various problem specifications with less data reliance. Concretely, the core architecture is built on pre-training and fine-tuning. Low-rank adaptation and the mixture of experts are further integrated to improve the adaptation ability. Experimental results demonstrate that on multiple vehicle routing problems, such as the traveling salesman problem and the capacitated vehicle routing problem, our framework achieves promising solution quality and computational efficiency.

Rui-Bin Zeng, Mo-Xuan Yang, and Ming-Long Lei contributed equally to this work. This research was supported in part by the National Natural Science Foundation of China (Nos. 12271503 and 11991021), and in part by the Strategic Priority Research Program of Chinese Academy of Sciences (No. XDA27010101).

✉ Ling-Feng Niu
niufl@ucas.ac.cn

¹ School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100049, China

² School of Mathematical Sciences, University of Chinese Academy of Sciences, Beijing 100049, China

³ College of Computer Science, Beijing University of Technology, Beijing 100124, China

⁴ Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing 100190, China

Keywords Efficient fine-tuning · Vehicle routing problems · Large language models · Combinatorial optimization

Mathematics Subject Classification 90C27 · 68T07 · 68T20

1 Introduction

Vehicle routing problem (VRP) is the whole set of routing planning problems, such as traveling salesman problem (TSP), capacitated VRP (CVRP), split delivery VRP (SDVRP), and prize collecting TSP (PCTSP) [42]. As a typical type of combinatorial optimization (CO) problem, VRPs appear frequently in practical events and have been researched for several decades in both optimization and artificial intelligence research fields. The main goal of these VRPs is to find the optimal set of routes for a fleet of vehicles to deliver goods to a set of customers while minimizing total transportation costs [28]. Related research can benefit many real-world applications in numerous ways, leading to higher efficiency and much greater cost savings.

Solving VRPs is extremely challenging due to their discrete property and inconstant real-world constraints [30]. Researchers have devoted considerable efforts to developing exact and heuristic algorithms. Mainstream exact algorithms include branch and bound methods [29], cutting plane [2], and branch and cut methods [35]. As the size of these problems grows, these traditional exact algorithms will experience significant computational increases. On the other hand, heuristic algorithms, e.g., ant colony optimization [8], genetic algorithm [1], Tabu search [11], provide efficient and problem-specific approximation solutions by trading optimality for computational speed and scalability.

With the development of computing equipment and data collections, learning-based methods have become an effective tool for solving combinatorial optimization problems, especially for vehicle routing problems. Tracing back to 1982, physicist John J. Hopfield proposed a neural network for associative memory, which is known as the Hopfield network [13], and achieved advanced results at that time. Generally, modern deep learning models train offline on instances of VRPs and then use the trained models to solve VRPs online. By extracting and utilizing the hidden features in these instances, the efficiency of solving and the quality of solutions can be improved. Typically, Vinyals et al. [43] proposed the Pointer Networks to solve VRPs, demonstrating that pure data-driven methods can directly solve VRPs and obtain approximate solutions. Kool et al. exploited the attention mechanisms in an encoder-decoder framework for feature extraction and routing construction [25]. Shi et al. [39] proposed the capsule aggregated attention mechanism, which utilizes capsules to store more information and applies dynamic routing for information aggregation. Pan et al. [36] unified the representation of various CO problems by simplifying them into general TSP problems to better transfer the knowledge in different VRPs.

Existing methods are still challenging and can be improved in the following aspects. First, most current approaches rely heavily on large-scale datasets for training, which may not always be available or practical, especially for newly emerging VRP variants. Although large language models (LLMs) have been initially explored for use in CO

problems [17–19, 37, 45], how to effectively integrate them with existing backbones remains unexplored and requires further research. Furthermore, the encoding process for VRPs is challenging to fully consider the node representation, the objective function, and the constraints. It is difficult to generalize well across different types of VRPs with a unified encoding process without extensive retraining. To address these challenges, we focus on developing a more flexible and data-efficient framework with LLMs that can capture the essential characteristics of VRPs in a unified manner, while also being capable of adapting to diverse problem specifications with minimal data requirements.

Therefore, we propose a universal framework for vehicle routing problems with LLMs in this paper. The core idea is to develop a generalizable and transferable pre-training and fine-tuning framework combined with LLMs for universal feature representation. To this aim, it is beneficial for different VRPs to form a universal representation through the feature extraction procedure. A common approach is to utilize the strong representation ability of LLMs through their pre-training on massive amounts of text data. Node feature representation combined with the assistance of LLMs can inspire the representational potential of existing models. Based on this universal representation, we further adopt a pre-training stage and a fine-tuning stage to extract general knowledge and explore the transferability between different VRPs. Our framework can be adaptive to VRPs with different objective functions and constraints. Besides, our method can jointly embed various VRP instances into shared features and is also suitable for various backbone architectures.

Concretely, we first use LLMs to encode the problem descriptions of VRPs into task embeddings, which serve as a universal support for these problems. Meanwhile, for each node, we transform its original locations into initial node embeddings. Then, we concatenate these initialized node embeddings with the task embeddings and feed the combined features into an encoder. The encoder extracts features from the input problem instances and task descriptions, while the decoder generates solutions step-by-step based on the output of the encoder. During the pre-training stage, we collect a large number of instances of arbitrary VRPs and use them to train the encoder-decoder architecture. The model learns the common knowledge of VRPs by capturing the characteristics and patterns in these instances. In the fine-tuning stage, we use low-rank adaptation (LoRA) [16] for fine-tuning and propose a modified version for the attention layers. We also adopt the mixture of experts (MoE) module [10] in both pre-training and fine-tuning stages to enhance the adaptability to different problems.

To summarize, the contributions of this work are as follows:

- (1) We propose a universal learning framework for VRPs with LLMs that can extract generalizable and transferable information to better solve VRPs.
- (2) We propose to use LLMs to support universal representations for different VRPs, by which all the problems can be represented in a unified form. The LLMs can provide extra global information according to problem settings.
- (3) We design a pre-training and fine-tuning architecture to extract generalizable and transferable knowledge generated from various VRPs and can adapt to a specific problem. This framework is versatile for various VRPs and can be used to boost the ability to solve these problems.

- (4) We evaluate our method on both synthetic datasets and open benchmarks. The numerical results demonstrate the effectiveness of our framework in solving VRPs.

2 Related Work

2.1 Learning Methods for VRPs

Machine learning techniques automatically capture heuristics for solving NP-hard CO problems. VRP is a typical class of CO problems with the aim of generating sequences as routing solutions. These methods that constructively generate routing solutions are called constructive methods. Some research focuses on applying and improving neural network models for solving VRPs, which can leverage the powerful representation capabilities of artificial intelligence and are recently underpinned by attention mechanisms. Another category of methods is concerned with learning strategies in reinforcement learning.

Neural networks for VRPs have shown significant ability in capturing the complex dependencies between nodes. This capability allows them to effectively model the intricate relationships and interactions among various stops or locations within a routing scenario. By understanding how the demand at one node affects the decisions and constraints related to other nodes, neural networks can generate more optimized and efficient routing plans. Pointer networks [43] constructively generate permutation sequences to formulate routing solutions. [3] turned it into deep reinforcement learning. [25] reinvented the pointer network by using a Transformer, which is named as attention model (AM) and has become a standard architecture for solving VRPs. By extending AM into changeable variants, several challenges within the field of vehicle routing have been addressed. In recent studies, there has been a notable emphasis on assessing the feature expressiveness concerning both scale shift [14, 41] and distributional shift [5, 9, 20, 32, 33, 46].

In addition to constructive solution generation, other studies have focused on solving VRPs by iteratively refining solutions, which are considered as improvement solvers. For instance, [6, 22, 31] have utilized local solvers to rewrite partial tours to enhance the solution quality. Moreover, some research has employed deep learning to train existing local search solvers. These methods include the 2-opt heuristic [34, 44], large neighborhood search [15], iterative dynamic programming [26], ant colony sampling [23], and the Lin-Kernighan-Helsgaun (LKH) algorithm. Additionally, several studies have explored fine-tuning schemes that aim at test-time adaptation within iterative learning [7, 14]. While constructive solvers are essential for rapidly generating initial feasible solutions, improvement solvers are vital for refining these solutions to achieve better optimality. These two approaches are fundamentally different and orthogonal in their objectives.

2.2 Large Language Models for CO

With the development of LLMs, increasing research efforts have explored their application to CO problems via automatic code generation and problem representation. Existing studies, such as FunSearch [37], demonstrate the capabilities of LLMs in mathematical discovery and general-purpose code search. They also achieve breakthroughs for problems like cap set discovery and improve solutions to classical combinatorial tasks such as the bin packing problem (BPP). ReEvo [45] has harnessed LLM-based generation and evolutionary search, enabling the direct construction of high-performance heuristics for real-world COPs, including TSP, CVRP, orienteering problem (OP), MKP, and BPP, and automating the development of meta-heuristics. However, these works mainly focus on individual variants or task-specific heuristics and lack a unified approach for transferable representation learning and generalization across different VRPs. Huang et al. [17] systematically investigated the ability of LLMs to solve a wide range of VRPs with natural language task descriptions. Their study introduces a comprehensive dataset covering 21 types of single- and multi-vehicle routing problems and evaluates LLMs across multiple prompt paradigms for text-to-code generation. Additionally, they propose a self-reflection framework to further enhance solution quality through iterative self-debugging and self-verification, demonstrating significant improvements in feasibility, optimality, and efficiency.

Another line of research leverages LLMs as semantic encoders for instances of CO problems, shifting from traditional structured representations toward the direct use of natural language and complex contextual information as input. In this paradigm, LLMs map instances of CO problems and constraints, described in plain language, into a high-dimensional and unified semantic vector space. These semantic embeddings are subsequently combined with neural solver architectures, such as Transformers or customized decoders, to enable end-to-end solution generation for diverse and previously unseen optimization tasks. These approaches overcome conventional limitations imposed by rigid data formats, fostering greater generalization to a broader array of problem types, sizes, and operational complexities. For instance, Jiang et al. [18] introduced a method focusing on the sequencing optimization of design structure matrices (DSM). This method combines LLM-derived semantic understanding of network topologies and domain-specific knowledge to generate novel combinatorial solutions in engineering design. Similarly, Jiang et al. [19] proposed the language-based neural COP solver (LNCS), which uses LLMs to encode both problem descriptions and instance features into unified semantic representations, and then integrates these embeddings with a Transformer-based solution generator. Their framework achieves state-of-the-art results on a range of CO problems, including TSP, CVRP, KP, simple multiple traveling workers problem (SMTWTP), and minimum vertex cover (MVC), which demonstrates both the effectiveness and generalizability of LLM-driven semantic representation. In contrast to these semantic encoding frameworks, our work is distinguished by proposing a fully unified, extensible pipeline specifically tailored for VRPs, integrating LLM-based semantic encoding, universal feature extraction, and transfer learning via pre-training and fine-tuning to address a broad range of VRP

variants under practical and large-scale constraints. The theoretical motivation for integrating LLMs into VRP solvers lies in their ability to bridge the gap between structural graph data and abstract task constraints. While specialized GNNs excel at capturing spatial relationships, they often lack the capacity to encode diverse logic-based constraints (e.g., prize-collecting rules) into a unified representation. By leveraging LLMs as semantic encoders, we map disparate VRP specifications into a shared high-dimensional semantic space. This enables the model to utilize the pre-trained reasoning capabilities of LLMs as a universal prior, facilitating zero-shot or few-shot transfer across routing variants.

3 Methodology

3.1 Preliminaries

An instance s of VRPs is denoted as a graph $\mathcal{G}$ with n nodes, where node $i \in \{1, 2, \dots, n\}$ has a specific location represented as $\mathbf{x}_i \in \mathbb{R}^2$ denoting the 2D coordinates. We define a stochastic policy $p(\pi | s)$ for selecting a routing solution π given the problem instance s . The encoder generates embeddings for all input nodes, while the decoder outputs a sequence π of these input nodes following the ranking order of the factorized and parameterized $p_\theta(\pi | s)$, where nodes are processed one by one.

The proposed framework contains a pre-training stage and a fine-tuning stage. In the pre-training stage, an encoder-decoder architecture is developed to extract universal knowledge from various VRP tasks. The fine-tuning stage further adapts the model to different VRP tasks with trainable MoE and LoRA modules.

3.2 Pre-training with Arbitrary VRP instances

In this subsection, we introduce our pre-training framework based on LLMs and the attention mechanism for diverse routing problems as presented in Fig. 1. We do not impose restrictions on the types of instances, where most VRP instances can be used for pre-training. We adopt an encoder-decoder architecture, where the encoder is responsible for compressing the input information into a fixed-length vector representation. The decoder generates an output sequence based on this vector representation (the solution to the problem), such as the node visiting order in TSP and the travel loops in CVRP.

3.2.1 Initialization

For VRP instances, each input node i is represented by the feature $\mathbf{x}_i$ with the dimension d_i (in most cases $d_i = 2$). We first transform the input feature $\mathbf{x}_i$ into a d -dimensional initial node embedding $\mathbf{h}_i^{\text{ini}}$ through linear projection:

$$\mathbf{h}_i^{\text{ini}} = \mathbf{W}^x \mathbf{x}_i + \mathbf{b}^x, \quad (1)$$

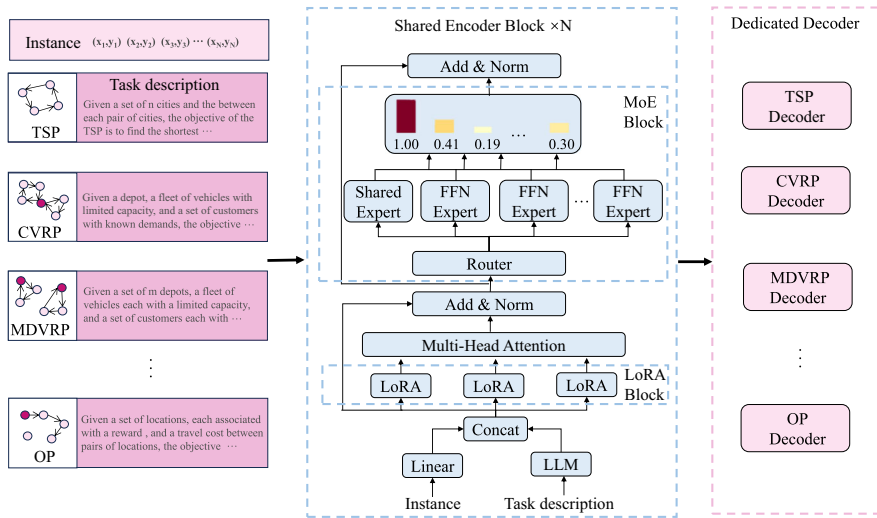


Fig. 1 Pre-training stage. The framework begins with task descriptions and instances that are fed into a series of shared encoder blocks, which include multi-head attention and MoE blocks. The MoE block comprises several feed-forward network (FFN) experts, managed by a router that assigns the tasks to the appropriate experts. Each task is then processed by its dedicated decoder

where $\mathbf{W}^x$ and $\mathbf{b}^x$ are learnable parameters. Then the initialized embedding is fed into the encoder and updated with L layers. Each layer is composed of an attention-based sublayer and a feed-forward sublayer.

Then, LLM is used to generate task context information by fully leveraging historical experience, which provides exclusive additional and task-specific information. Task context plays an auxiliary role in the model's training stage, facilitating the model to learn and understand the knowledge and characteristics related to the VRP more efficiently. We encode the task description text, denoted as $\tilde{\mathbf{X}}$, by using an arbitrary LLM-based model $\text{LLM}(\cdot)$ to obtain a fixed-length task embedding vector $\tilde{\mathbf{h}} \in \mathbb{R}^{1 \times d}$ where $\tilde{\mathbf{W}}$ denotes the parameters of the LLM for model control. This process can be formally expressed as

$$\tilde{\mathbf{h}} = \text{LLM}(\tilde{\mathbf{X}}, \tilde{\mathbf{W}}). \quad (2)$$

This vector captures the semantic information of the specific VRP task. We concatenate the node embeddings with the task embedding $\tilde{\mathbf{h}}$ to form the initial input representation $\mathbf{H}^{(0)}$ for the encoder:

$$\mathbf{H}^{(0)} = [\mathbf{h}_{(0)}^{\text{ini}}, \dots, \mathbf{h}_{(n-1)}^{\text{ini}}, \tilde{\mathbf{h}}]. \quad (3)$$

3.2.2 Encoder

The encoder extracts high-level features with a transformer network from the concatenated input representation $\mathbf{H}^{(0)}$. More generally, in each layer l , we project the

embedding $\mathbf{H}^{(l)}$ into three representations $\mathbf{Q}^{(l)}$, $\mathbf{K}^{(l)}$, and $\mathbf{V}^{(l)}$:

$$\mathbf{Q}^{(l)} = \mathbf{H}^{(l)} \mathbf{W}_{\mathbf{Q}}^{(l)}, \mathbf{K}^{(l)} = \mathbf{H}^{(l)} \mathbf{W}_{\mathbf{K}}^{(l)}, \mathbf{V}^{(l)} = \mathbf{H}^{(l)} \mathbf{W}_{\mathbf{V}}^{(l)}, \quad (4)$$

where $\mathbf{Q}^{(l)}$, $\mathbf{K}^{(l)}$, and $\mathbf{V}^{(l)}$ are projected representations by their corresponding parameter matrices $\mathbf{W}_{\mathbf{Q}}^{(l)} \in \mathbb{R}^{d \times d}$, $\mathbf{W}_{\mathbf{K}}^{(l)} \in \mathbb{R}^{d \times d}$, and $\mathbf{W}_{\mathbf{V}}^{(l)} \in \mathbb{R}^{d \times d}$. The attention scores can be calculated as

$$\text{ATT}(\mathbf{H}^{(l)}) = \text{softmax} \left(\frac{\mathbf{Q}^{(l)} \mathbf{K}^{(l)\top}}{\sqrt{d_k}} \right) \mathbf{V}^{(l)}, \quad (5)$$

where $\text{ATT}(\cdot)$ is the attention operation. In a general graph, the attention score of non-adjacent nodes is set to $-\infty$ to prevent message-passing between them.

The above formula can be extended to a multi-head attention mechanism that generates multiple representations in different subspaces independently. Denote the i -th attention head as $\text{head}_i = \text{ATT}(\mathbf{H}^{(l)})$, the results after multi-attention can be represented as

$$\mathbf{H}'^{(l)} = \parallel_{m=1}^M \text{head}_M \mathbf{W}_{\mathbf{O}}^{(l)}, \quad (6)$$

where $\parallel_{m=1}^M$ is the vertically sequential concatenation operation, $\mathbf{W}_{\mathbf{O}}^{(l)}$ is a parameter matrix, and $\mathbf{H}'^{(l)}$ denotes the features after multi-head attention. After the multi-head attention layer, we apply the layer normalization and feed-forward layers with non-linear transformations to obtain updated node feature $\mathbf{H}^{(l+1)}$:

$$\mathbf{H}^{(l+1)} = \text{Linear} \left(\text{ReLU} \left(\text{Linear}(\mathbf{H}'^{(l)}) \right) \right). \quad (7)$$

We further introduce the MoE as a modified version of the FFN module in both pre-training and fine-tuning stages. For clear demonstrations, we present the original FFN above and introduce the detailed MoE version in Sect. 3.3.2.

After being processed through L layers of encoding, the encoder generates a global graph embedding $\bar{\mathbf{h}}^{(L)}$, which represents the global characteristics of the entire problem instance. It is specifically defined as the mean of all node embeddings:

$$\bar{\mathbf{h}}^{(L)} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i^{(L)}, \quad (8)$$

where $\mathbf{h}_i^{(L)}$ denotes the embedding vector of the i -th node at the L -th layer, and n is the total number of nodes. This global embedding aggregates information from all nodes and captures overall properties of the graph structure, which is later combined with node embeddings in the decoder to facilitate context-aware routing decisions.

3.2.3 Decoder

While VRP variants remain NP-hard, the complexity of learning an effective constructive policy is largely dictated by the semantic gap between raw coordinates and

abstract task constraints. By leveraging LLMs to map task descriptions into a unified semantic space, we use a decoder with explicit functional contexts. This does not reduce the problem's inherent hardness but significantly improves the signal-to-noise ratio in the feature representation, allowing the attention-based decoder to identify high-quality routing patterns more efficiently without the need for exhaustive computational search.

The decoder outputs the node π_t based on the encoder's embeddings and the outputs $\pi_{t'}$ generated at time $t' < t$. During the decoding process, we use the $\mathbf{h}_c^{(l)}$ to represent the context node embeddings. These context embeddings for different VRPs are not totally the same. We follow the context design in [4, 25] and give two examples of PCTSPs and CVRPs.

The output of the PCTSPs at time t is the current/last location π_{t-1} and the remaining prize to collect $\mathbf{p}_t$. Again, we do not need placeholders if $t = 1$ as the route starts at the depot, and we do not need to provide information about the first node as the route should end at the depot. The information about the prizes collected is implicitly provided to the model in the form of $\mathbf{p}_t$, and we do not need to provide any information about the penalties as this is irrelevant for the remaining decisions:

$$\mathbf{h}_c^{(l)} = \begin{cases} [\bar{\mathbf{h}}^{(l)}, \mathbf{h}_{\pi_{t-1}}^{(l)}, \mathbf{p}_t], & \text{if } t > 1, \\ [\bar{\mathbf{h}}^{(l)}, \mathbf{h}_0^{(l)}, \mathbf{p}_t], & \text{if } t = 1. \end{cases} \quad (9)$$

The context for the decoder for the VRPs at time t is the current/last location π_{t-1} and the remaining capacity $\hat{D}_t$. Compared to TSPs, we do not need placeholders if $t = 1$ as the route starts at the depot and we do not need to provide information about the first node as the route should end at the depot:

$$\mathbf{h}_c^{(l)} = \begin{cases} [\bar{\mathbf{h}}^{(l)}, \mathbf{h}_{\pi_{t-1}}^{(l)}, \hat{\mathbf{d}}_t], & \text{if } t > 1, \\ [\bar{\mathbf{h}}^{(l)}, \mathbf{h}_0^{(l)}, \hat{\mathbf{d}}_t], & \text{if } t = 1. \end{cases} \quad (10)$$

The context of the decoder at time t comes from the encoder and the output up to time t . The new context node embedding $\mathbf{h}_{(c)}^{(l+1)}$ is calculated through the attention mechanism. The keys and values are derived from the node embeddings $\mathbf{h}_i^{(l)}$, and the query $\mathbf{q}_c$ is calculated from the context node:

$$\mathbf{q}_c^{(l)} = \mathbf{W}_Q^{(l)} \mathbf{h}_c^{(l)}, \quad \mathbf{k}_i^{(l)} = \mathbf{W}_K^{(l)} \mathbf{h}_i^{(l)}, \quad \mathbf{v}_i^{(l)} = \mathbf{W}_V^{(l)} \mathbf{h}_i^{(l)}. \quad (11)$$

The attention score between the query and all nodes is calculated, and masking is performed on the nodes that cannot be accessed (i.e., the nodes that have already been visited):

$$a_{c,j}^{(l)} = \begin{cases} \frac{\mathbf{q}_c^{(l)\top} \mathbf{k}_j^{(l)}}{\sqrt{d_k}}, & \text{if } j \neq \pi_{t'} \text{ for all } t' < t, \\ -\infty, & \text{otherwise,} \end{cases} \quad (12)$$

where $a_{c,j}$ is the calculated attention score. Finally, the new context node embedding $\mathbf{h}_c^{(l+1)}$ is obtained through the attention calculation method similar to the encoder in

Eqs. (6) and (7). To calculate the output probability $p_\theta(\pi_t | s, \pi_{1:t-1})$, the decoder adds a final layer that uses a single-head attention mechanism. When calculating the final attention scores, the result is first clipped (before masking) within the range $[-C, C]$ using the tanh function:

$$a'_{c,j} = \begin{cases} C \cdot \tanh(\frac{q_c^\top k_j}{\sqrt{d_k}}), & \text{if } j \neq \pi_{t'} \text{ for all } t' < t, \\ -\infty, & \text{otherwise.} \end{cases} \quad (13)$$

These attention scores are interpreted as unnormalized log-probabilities (logits), and the final output probability vector is calculated using the softmax function:

$$p_i = p_\theta(\pi_t = i | s, \pi_{1:t-1}) = \frac{\exp^{a'_{c,i}}}{\sum_j \exp^{a'_{c,j}}}. \quad (14)$$

The decoder sequentially outputs nodes π_t step by step based on the embedding of the encoder and the sequence of nodes already generated. During the training phase, we optimize the model parameters θ using the reinforcement learning algorithm, whose main objective is to minimize the expected loss function $\mathcal{L}(\theta | s)$. In other words, the objective is to minimize the expected difference between the solution cost $L(\pi)$ and the baseline $b(s)$ as reflected in the gradient estimate of the loss function:

$$\mathcal{L}_{\text{Reinforce}} = \mathbb{E}_{p_\theta(\pi | s)} [(L(\pi) - b(s)) \nabla \log p_\theta(\pi | s)], \quad (15)$$

where $p_\theta(\pi | s)$ denotes the probability distribution of the generation of solution π by the model, and $L(\pi)$ is the cost of the solution and $b(s)$ is the rollout baseline, which estimates the difficulty of the problem instance s to reduce the variance of the gradient. The baseline $b(s)$ is computed using a greedy rollout strategy with the current model, at each decoding step, the node with the highest probability is selected deterministically to construct the baseline solution π_{BL} , and its cost $L(\pi_{\text{BL}})$ serves as the baseline value.

3.3 Fine-Tuning with Low-Rank Adaptation

In this subsection, we propose a fine-tuning architecture that integrates LoRA and MoE to enhance adaptability to diverse VRP tasks. As illustrated in Fig. 2, the architecture encompasses an end-to-end workflow for both pre-training and fine-tuning stages: problem instances and task descriptions are embedded and concatenated, then processed by multi-head attention layers with LoRA modules and MoE-based feed-forward networks, where a router mechanism enables dynamic expert selection during inference. During pre-training, all components, including Add and Norm, Linear, Multi-Head Attention, MoE FFN, Router, and LoRA layers, are trained to ensure broad generalization. In fine-tuning, certain layers are frozen, while LoRA and MoE-related parameters remain adaptive, achieving task-specific optimization with minimal parameter updates.

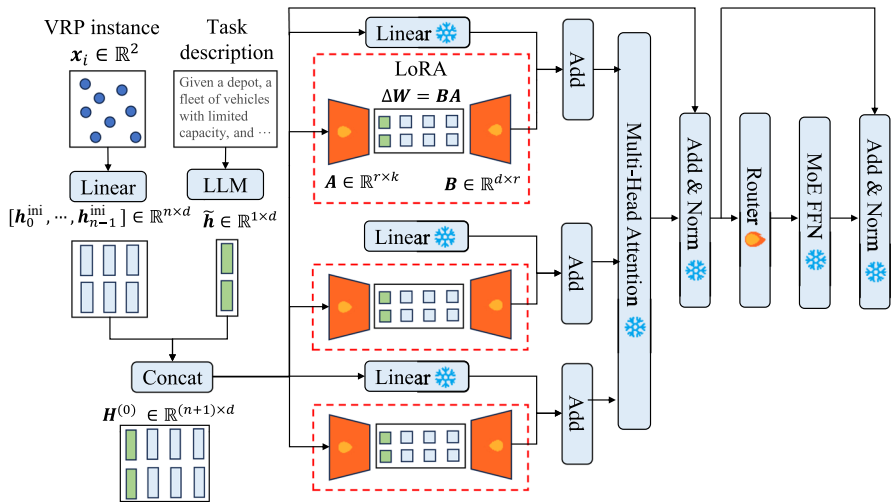


Fig. 2 Universal framework for VRPs. In the pre-training stage, all components are subject to weight updates, including the Add and Norm, MoE FFN, Router, Multi-Head Attention, Linear, and LoRA layers. The instance and task descriptions are processed through a series of layers before being concatenated. In the fine-tuning stage, certain layers, such as the Add, Norm, and Linear, are frozen, preventing weight updates during fine-tuning. This selective freezing is applied to the Multi-Head Attention and MoE FFN blocks as well

3.3.1 Low-Rank Adaptation

In the fine-tuning stage, we use LoRA for fine-tuning. We reconsider the generation of attention weights in the attention layers. Our model contains several dense layers that perform matrix multiplication. The weight matrices in these layers typically have full rank. When adapting to a specific task, the pre-trained models have a low intrinsic dimension and can still learn efficiently despite a random projection to a smaller subspace. We hypothesize the updates to the weights also have a low intrinsic rank during adaptation. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d \times k}$, we constrain its update by representing the latter with a low-rank decomposition $W_0 + \Delta W = W_0 + BA$, where $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$, and the rank $r \ll \min(d, k)$. During training, W_0 is frozen and does not receive gradient updates, while A and B contain trainable parameters. Note that both W_0 and BA are multiplied with the same input, and their respective output vectors are summed coordinate-wise. For Eq. (4), we take Query (Q) as an example, our modified version is

$$Q^{(l)} = W_Q^{(l)} H^{(l)} + \Delta W_Q^{(l)} H^{(l)} = W_Q^{(l)} H^{(l)} + (B_Q^{(l)} A_Q^{(l)}) H^{(l)}. \quad (16)$$

Similar operations can be applied to Key (K) and Value (V). Gaussian is used for initializing A , and zeros are used for initializing B . We then scale $\Delta W_Q^{(l)} H^{(l)}$ by $\frac{\alpha}{r}$, where α is a constant in r . We follow the original LoRA that sets α to the first r we try without tuning. We do not have to retune hyperparameters when we vary r with the scaling operation.

A more general form of fine-tuning allows the training of a subset of the pre-trained parameters. LoRA takes a step further and does not require the accumulated gradient update to weight matrices to have full rank during adaptation. This means that when applying LoRA to all weight matrices and training all biases, we roughly recover the expressiveness of full fine-tuning by setting the LoRA rank r to the rank of the pre-trained weight matrices. In other words, as we increase the number of trainable parameters, training LoRA roughly converges to training the original model, while adapter-based methods converge to an MLP and prefix-based methods to a model that cannot take long input sequences.

$\mathbf{W}_Q^{(l)}$, $\mathbf{W}_K^{(l)}$, and $\mathbf{W}_V^{(l)}$ can be explicitly computed and used for testing. When we need to switch to another VRP task, we can recover $\mathbf{W}_Q^{(l)}$, $\mathbf{W}_K^{(l)}$, and $\mathbf{W}_V^{(l)}$ by subtracting $\mathbf{B}\mathbf{A}$ and then adding a different $\mathbf{B}'\mathbf{A}'$. This is a quick operation with very little memory overhead. Critically, this guarantees that we do not introduce any additional latency during inference compared to a fine-tuned model by construction. We can apply LoRA to the weight matrix to reduce the number of trainable parameters. In the attention-based architecture, we treat $\mathbf{W}_Q$ (or $\mathbf{W}_K$, $\mathbf{W}_V$) as a one matrix of dimension $d \times d$. The attention weights are adapted for downstream tasks for simplicity and efficiency.

3.3.2 Mixture of Experts on FFNs

We also introduce the MoEs to enhance the adaptability to different problems. A MoE module takes as an input an nodes representation $\mathbf{H}'^{(l)}$ and then routes this to the best determined top- k experts, selected from a set $\{E_i(\mathbf{H}'^{(l)})\}_{i=1}^\gamma$ of γ experts. $E_i(\cdot)$ is the i -th expert's feed-forward operation whose details are same as Eq. (7). The router variable $\mathbf{W}_r$ produces logits $\delta^{(l)} = \mathbf{W}_r^{(l)} \cdot \mathbf{H}'^{(l)}$ which are normalized via a softmax distribution over the available γ experts at that layer. The top- k gate values are selected for routing the nodes x . If $\mathcal{T}$ is the set of selected top- k indices, then the output computation of the layer is the linearly weighted combination of each expert's computation on the nodes by the gate value,

$$\mathbf{y}^{(l)} = \sum_{i \in \mathcal{T}} \frac{\exp^{\delta_i^{(l)}}}{\sum_j^\gamma \exp^{\delta_j^{(l)}}} E_i(\mathbf{H}'^{(l)}). \quad (17)$$

To encourage a balanced load across experts, we add an auxiliary loss. For each encoder layer, this auxiliary loss is added to the total model loss during training. Given γ experts and a batch $\mathcal{B}$ with n nodes, the auxiliary loss is

$$\mathcal{L}_{\text{aux}} = \gamma \cdot \sum_{i=1}^\gamma \frac{1}{n} \sum_{x \in \mathcal{B}} \mathbf{1}\{\text{argmax } \zeta(x) = i\} \cdot P_i, \quad (18)$$

where f_i is the fraction of nodes dispatched to expert i , P_i is the fraction of the router probability allocated for expert i , and $\zeta_i(x)$ is the probability of routing nodes x to expert i . In addition, we introduce an indicator function $\mathbf{1}$ to denote whether a particular

node is routed to the i -th expert. P_i is the probability fraction to expert i in the batch $\mathcal{B}$.

$$P_i = \frac{1}{n} \sum_{x \in \mathcal{B}} \frac{\exp^{\delta_i^{(l)}}}{\sum_j \exp^{\delta_j^{(l)}}}. \quad (19)$$

When the routing is completely uniform, the minimal loss value is $\mathcal{L}_{\text{aux}} = 1$. If some experts deviate, the loss will increase accordingly, thereby encouraging the router to adjust its routing policy toward a more uniform distribution. The value P_i is computed from the router's softmax output and is differentiable, whereas the indicator function $\mathbf{1}$ is determined by discrete routing decisions via $\arg \max$, making it non-differentiable. Consequently, the loss function leverages the gradients with respect to P_i to indirectly update the router parameters, guiding it to produce a more uniform probability distribution.

Note that the auxiliary loss is only used in the pre-training stage, where the overall loss is formulated as $\mathcal{L}_p = \mathcal{L}_{\text{Reinforce}} + \alpha \mathcal{L}_{\text{aux}}$. The hyperparameter α serves as a multiplicative coefficient for the auxiliary loss. To balance the gradient contributions between the auxiliary loss $\mathcal{L}_{\text{aux}}$ and the reinforcement loss $\mathcal{L}_{\text{Reinforce}}$, we set $\alpha = 0.1$. This choice enables rapid load balancing without adversely affecting the overall training process.

3.4 Overall Algorithms

Most previous works regard specific constraints of VRPs as additional inputs for training. However, we extract task-specific context according to LLMs. Furthermore, we extract the universal feature representation with arbitrary VRP instances in the pre-training stage and adapt to the specific VRP problem in the fine-tuning stage. The overall algorithms for pre-training and fine-tuning are presented in Algorithm 1 and Algorithm 2.

4 Experiments

In this section, we present experimental results of our universal framework for VRPs with LLMs on several routing problems. We fully demonstrate the effectiveness and efficiency of the model with overall performance comparison, ablation study, parameter analysis, and expert activation visualization.

Algorithm 1 Pre-training stage with LLM-enhanced universal framework**Input:** Number of layers L Number of iterations $iters$.**Output:** Pre-trained parameters Θ_{enc} .

```

1 Initialize node embeddings  $\mathbf{h}_i^{(0)} \leftarrow \text{Eq. (1)}$ ;
2 Encode task description  $\tilde{\mathbf{h}} \leftarrow \text{Eq. (2)}$ ;
3 Concatenate node and task embeddings  $\mathbf{H}^{(0)} \leftarrow \text{Eq. (3)}$ ;
4 for  $iter \leq iters$  do
5   for  $l \leq L$  do
6     Compute  $\mathbf{H}^{(l)}$  via multi-head attention  $\mathbf{H}^{(l)} \leftarrow \text{Eqs. (4) (5) (6)}$ ;
7     Compute MoE-FFN output  $\mathbf{y}^{(l)} \leftarrow \text{Eq. (17)}$ ;
8   end for
9   Extract global graph context  $\bar{\mathbf{h}}^{(L)} \leftarrow \text{Eq. (8)}$ ;
10  for  $t = 1$  to  $n$  do
11    for  $l \leq L$  do
12      Compute context embedding  $\mathbf{h}_c^{(l)} \leftarrow \text{Eq. (9)}$ ;
13      Apply masking and compute attention score  $a_{c,j}^{(l)} \leftarrow \text{Eq. (12)}$ ;
14      Compute node probabilities  $p_i \leftarrow \text{Eq. (14)}$ ;
15    end for
16  end for
17  Compute loss  $\mathcal{L}_p = \mathcal{L}_{\text{Reinforce}} + \alpha \cdot \mathcal{L}_{\text{aux}} \leftarrow \text{Eqs. (15) (18)}$ ;
18  Backward and update parameters;
19 end for
20 Obtain the updated parameters  $\Theta_{enc}$ .
```

Algorithm 2 Fine-tuning stage**Input:** Pre-trained parameters Θ_{enc} ,Number of layers L ,Number of LoRA rank r ,Number of fine-tuning iterations $iters$.**Output:** Fine-tuned parameters $\Theta_{\text{LoRA}}, \Theta_{\text{router}}$ for LoRA adapters and MoE router.

```

Initialize node embeddings  $\mathbf{h}_i^{(0)} \leftarrow \text{Eq. (1)}$ ;
1 Encode task description  $\tilde{\mathbf{h}} \leftarrow \text{Eq. (2)}$ ;
2 Concatenate node and task embeddings  $\mathbf{H}^{(0)} \leftarrow \text{Eq. (3)}$ ;
3 for  $iter \leq iters$  do
4   for  $l \leq L$  do
5     Compute  $\mathbf{H}^{(l)}$  via modified multi-head attention with LoRA  $\leftarrow \text{Eqs. (16) (5) (6)}$ ;
6     Compute MoE-FFN output  $\mathbf{y}^{(l)} \leftarrow \text{Eq. (17)}$ ;
7     Extract global graph context  $\bar{\mathbf{h}}^{(L)} \leftarrow \text{Eq. (8)}$ ;
8   end for
9   for  $t = 1$  to  $n$  do
10    for  $l \leq L$  do
11      Compute context embedding  $\mathbf{h}_c^{(l)} \leftarrow \text{Eq. (9)}$ ;
12      Apply masking and compute attention score  $a_{c,j}^{(l)} \leftarrow \text{Eq. (12)}$ ;
13      Compute node probabilities  $p_i \leftarrow \text{Eq. (14)}$ ;
14    end for
15  end for
16  Compute loss  $\mathcal{L}_{pt} = \mathcal{L}_{\text{Reinforce}} \leftarrow \text{Eq. (15)}$ ;
17  Backward and update trainable parameters  $\Theta_{\text{LoRA}}$  and  $\Theta_{\text{router}}$ ;
18 end for
19 Return fine-tuned parameters  $\Theta_{\text{LoRA}}, \Theta_{\text{router}}$ .
```

4.1 Experimental Setup

4.1.1 Categories of VRPs

TSP: Given a set of cities and their locations, the objective of the TSP is to find the shortest possible route that visits each city exactly once and returns to the starting city.

CVRP: It involves designing optimal routes for a fleet of vehicles to deliver goods to a set of customers from a central depot, while respecting vehicle capacity constraints. The objective is to minimize the total transportation cost (e.g., distance or time) while ensuring that each customer's demand is met exactly once by a single vehicle.

OP: It involves selecting a subset of nodes (or points of interest, POIs) to visit and determining the order of these visits to maximize the total collected score, subject to a given time or distance budget.

PCTSP: In the PCTSP, each node has not only an associated prize, but also an associated penalty. The goal is to collect at least a minimum total prize, while minimizing the total tour length plus the sum of penalties of unvisited nodes. This problem is difficult as an algorithm has to trade-off the penalty for not visiting a node with the marginal cost/tour length of visiting (which depends on the other nodes visited), while also satisfying the minimum total prize constraint.

Stochastic Prize and Collection Traveling Salesman Problem (SPCTSP): As a stochastic variant of the PCTSP, we use SPCTSP to show how our model can deal with uncertainty naturally. In the SPCTSP, the expected node prize is known upfront, but the real collected prize only becomes known upon visitation. With penalties, this problem is a generalization of the stochastic k-TSP.

SDVRP: It determines a set of vehicle routes that minimize the total transportation cost while satisfying the vehicle capacity constraints. Specifically, vehicles depart from a depot, visit customer locations sequentially, and can collectively fulfill a customer's demand (which may exceed a single vehicle's capacity) by splitting the delivery among multiple vehicles. Finally, the vehicles return to the depot.

Vehicle Routing Problem with Time Windows (VRPTW) [40]: It extends the CVRP by associating a time window with each customer, representing the interval during which service must begin. The objective is to minimize the total transportation cost using a fleet of vehicles that start and end at a central depot. Each vehicle must serve a set of customers within their specified time windows and vehicle capacity constraints. If a vehicle arrives at a customer location before the time window opens, it must wait until the start time.

Pickup and Delivery Problem (PDP) [38]: It involves transporting goods between paired pickup and delivery locations using a fleet of vehicles. Unlike standard VRPs where goods are distributed from a central depot, requests in PDP consist of moving items from a specific origin node to a specific destination node. The objective is to construct a set of optimal routes that minimize the total cost while ensuring that for every request, the pickup location is visited before the corresponding delivery location by the same vehicle, and capacity constraints are respected throughout the route.

Multi-Trip Vehicle Routing Problem (MTVRP) [21]: It allows vehicles to perform multiple trips during a working period, relaxing the standard VRP assumption that each vehicle operates only one route. The objective is to minimize the total travel cost

or fleet size required to serve all customers. Vehicles depart from the depot, serve a subset of customers, return to the depot to reload, and then depart again for subsequent trips, provided that the total duration of all trips assigned to a vehicle does not exceed a maximum time limit or shift duration.

4.1.2 Baselines

To evaluate our method, we compare it with several baseline methods that have been widely used in solving VRPs. These methods can be roughly classified as exact solvers, heuristic methods, and learning-based methods. Concorde, LKH3 [12] and Gurobi are exact CO solvers that provide optimal solutions. OR-tools [3] is an approximate solver based on meta-heuristics. We compare our method against non-learned baselines that directly construct a tour in order, like Nearest, Random, and Farthest Insertion. We also present comparisons with various learned heuristics: AM [25], POMO [27], and Sym-NCO [24].

4.1.3 Experimental Settings

The framework was implemented in Python using PyTorch. All problem instances were generated with the RL4CO library. Experiments were conducted on a workstation equipped with eight NVIDIA RTX3090 GPUs, an Intel Xeon Silver 4310 CPU @ 2.10 GHz, and 521 GB of RAM. Feature dimensionality was set to 128. Both the encoder and decoder contain three layers. A hybrid MoE employed four experts. During the fine-tuning stage, LoRA was applied to all weight matrices, updating only the MoE routing layers. The pre-training stage is trained for a maximum of 300 epochs, followed by 100 epochs of fine-tuning. The Adam optimizer was used with an initial learning rate of 0.0001, incorporating a linear warm-up schedule and a per-epoch multiplicative decay factor of 0.98. All other hyperparameters were held constant across experiments.

Following the standard protocols prescribed in AM [25], POMO [27], and Sym-NCO [24], we generate training instances dynamically during training. The node coordinates for all problem variants are sampled from a uniform distribution $\mathcal{U}(0, 1)$ within the unit square. To strictly enforce a uniform distribution across tasks during our pre-training, we adopt a balanced generation strategy. Specifically, each training epoch consists of 10 000 instances for one of the six VRP variants (TSP, CVRP, OP, PCTSP, SDVRP, and SPCTSP), generating a total number of 60 000 instances per epoch. This strategy ensures that the model optimizes for all tasks equally. Consistent with previous studies, we conduct training separately for each problem scale ($n = 20, 50, 100$). For evaluation, we allow the model to solve standard fixed test sets with 10 000 instances to ensure fair comparisons.

4.2 Overall Results

We record the optimality gap and inference time for all methods on various VRPs at different scales, relative to optimal solutions obtained by solvers such as Concorde, LKH3, and Gurobi. The comparative results are reported in Table 1.

Table 1 Performance comparison of different approaches for VRPs on different scales. Obj. is the objective value, Gap (%) is the gap of the best value across all methods, and “-” denotes out of memory. The best performance is marked in bold

Methods	$n=20$			$n=50$			$n=100$		
	Obj	Gap(%)	Time	Obj	Gap(%)	Time	Obj	Gap(%)	Time
TSP	Concorde	3.84	0.00%	1 m	5.7	0.00%	7.76	0.00%	3 m
	LKH3	3.84	0.00%	18 s	5.7	0.00%	7.76	0.00%	21 m
	Gurobi	3.84	0.00%	7 s	5.7	0.00%	7.76	0.00%	17 m
	Nearest Insertion	4.33	12.91%	1 s	6.78	19.03%	9.46	21.82%	6 s
	Random Insertion	4.00	4.36%	<1 s	6.13	7.65%	8.52	9.69%	3 s
	Farthest Insertion	3.93	2.36%	1 s	6.01	5.53%	8.35	7.59%	7 s
	Nearest Neighbor	4.50	17.23%	<1 s	7.00	22.94%	9.68	24.73%	<1 s
	AM	3.84	0.01%	5 m	5.73	0.52%	7.94	2.26%	1 h
	POMO	3.84	0.00%	15 s	5.7	0.22%	7.80	0.48%	3 m
	Sym-NCO	3.84	0.00%	15 s	5.7	0.19%	7.79	0.39%	3 m
CVRP	AM(ours)	3.84	0.00%	5 m	5.71	0.24%	7.92	2.12%	1 h
	POMO(ours)	3.84	0.00%	16 s	5.7	0.13%	7.8	0.43%	3 m
	Sym-NCO(ours)	3.84	0.00%	16 s	5.7	0.08%	7.78	0.38%	3 m
	LKH3	6.14	0.00%	2 h	10.38	0.00%	15.65	0.00%	13 h
	OR-tools	6.43	5.41%	2 m	11.31	9.01%	17.16	9.67%	1 h
	AM	6.27	2.13%	6 m	10.62	2.40%	16.23	3.72%	2 h
	POMO	6.19	0.81%	10 s	10.49	1.15%	15.88	1.46%	6 m

Table 1 continued

Methods	n=20			n=50			n=100		
	Obj	Gap(%)	Time	Obj	Gap(%)	Time	Obj	Gap(%)	Time
OP	Sym-NCO	6.19	0.80%	10s	10.47	1.12%	15.85	1.28%	6m
	AM(ours)	6.26	2.06%	6m	10.61	2.26%	16.23	3.68%	2h
	POMO(ours)	6.19	0.79%	11s	10.49	1.11%	15.88	1.46%	6m
	Sym-NCO(ours)	6.18	0.76%	10s	10.47	1.09%	15.85	1.27%	6m
	Gurobi	5.39	0.00%	16m	—	—	—	—	—
	Gurobi (1s)	4.62	14.22%	4m	1.29	92.03%	0.58	98.25%	7m
	Gurobi (10s)	5.37	0.33%	12m	10.96	32.20%	1.34	95.97%	53m
	Gurobi (30s)	5.38	0.05%	14m	13.57	16.09%	3.23	90.28%	3h
	Compass	5.37	0.36%	2m	16.17	0.00%	33.19	0.00%	15m
	AM	5.30	1.56%	4m	16.07	0.60%	32.68	1.55%	53m
	POMO	5.32	1.14%	12s	16.11	0.37%	32.97	0.66%	6m
	Sym-NCO	5.32	1.12%	12s	16.14	0.16%	33.04	0.45%	6m
PCTSP	AM(ours)	5.31	1.44%	4m	16.07	0.58%	32.67	1.55%	53m
	POMO(ours)	5.32	1.04%	13s	16.11	0.35%	32.99	0.64%	7m
	Sym-NCO(ours)	5.34	0.96%	13s	16.14	0.15%	33.04	0.45%	7m
	Gurobi	3.13	0.00%	2m	—	—	—	—	—
	Gurobi (1s)	3.14	0.07%	1m	—	—	—	—	—
	Gurobi (10s)	3.13	0.00%	2m	4.54	1.36%	—	—	—

Table 1 continued

Methods	$n=20$			$n=50$			$n=100$		
	Obj	Gap(%)	Time	Obj	Gap(%)	Time	Obj	Gap(%)	Time
SDVRP	Gurobi (30s)	3.13	0.00%	2 m	4.48	0.03%	54 m	–	–
	ILS (C++)	3.16	0.77%	16 m	4.50	0.36%	2 h	5.98	0.00%
	ORTools (10s)	3.14	0.05%	52 m	4.51	0.70%	52 m	6.35	6.21%
	ORTools (60s)	3.13	0.01%	5 h	4.48	0.00%	5 h	6.07	1.56%
	AM	3.15	0.45%	5 m	4.52	0.74%	19 m	6.08	1.67%
	POMO	3.14	0.05%	12 s	4.49	0.22%	1 m	6.01	0.43%
	Sym-NCO	3.13	0.01%	13 s	4.48	0.08%	1 m	6.00	0.38%
	AM(ours)	3.15	0.38%	5 m	4.52	0.72%	19 m	6.07	1.15%
	POMO(ours)	3.14	0.05%	14 s	4.49	0.22%	1 m	6.01	0.43%
	Sym-NCO(ours)	3.13	0.00%	14 s	4.48	0.07%	1 m	5.99	0.26%
	AM	6.37	3.73%	9 m	10.91	6.05%	42 m	17.19	8.68%
	Sym-NCO	6.29	2.51%	5 s	10.47	1.78%	3 m	16.24	2.63%
SPCTSP	AM(ours)	6.36	3.71%	9 m	10.81	5.10%	42 m	17.12	8.19%
	Sym-NCO(ours)	6.14	0.00%	5 s	10.29	0.00%	3 m	15.82	0.00%
	AM	3.42	5.71%	5 m	5.28	16.35%	8 m	7.50	13.89%
	POMO	3.47	7.23%	7 s	5.07	11.60%	36 s	7.22	9.64%
	Sym-NCO	3.37	3.93%	7 s	4.89	7.66%	36 s	7.02	6.55%
	AM(ours)	3.41	5.42%	5 m	5.26	15.90%	8 m	7.46	13.30%
	POMO(ours)	3.41	5.43%	8 s	5.01	10.43%	37 s	7.14	8.42%
	Sym-NCO(ours)	3.24	0.00%	7 s	4.54	0.00%	37 s	6.59	0.00%

For TSP, the standard AM method achieves a 0.01% gap at $n = 20$, and the LLM-enhanced AM variant attains 0.00%. This trend continues at larger scales. When $n = 50$, the gap reduces from 0.52% to 0.24%, and from 2.26% to 2.12% when $n = 100$. Similarly, POMO integrated with the LLM consistently shows slightly lower gaps, as does Sym-NCO (e.g., 0.08% vs 0.19% at $n = 50$). These improvements in solution quality are achieved without a significant increase in computation time across all tested scales. The consistent marginal improvements on TSP across different methods (AM, POMO, Sym-NCO) suggest that the universal feature representation provided by large language models is beneficial for fundamental VRPs.

For CVRP, the AM variant enhanced by the LLM consistently achieves slightly lower optimality gaps compared to the standard AM, while maintaining the same inference time. The POMO variant also exhibits minor reductions in the optimality gap (e.g., 1.11% vs 1.15% when $n = 50$). Sym-NCO integrated with the LLM shows more pronounced reduction in gaps across all scales (e.g., 0.76% vs 0.80% at $n = 20$, 1.09% vs 1.12% at $n = 50$, 1.27% vs 1.28% at $n = 100$), with computation times comparable to the standard Sym-NCO. These results indicate that the framework's ability to capture problem settings via LLMs is transferable and beneficial for VRPs with different structural properties.

For OP, AM coupled with the LLM shows reduced gaps at $n = 20$ (1.44% vs 1.56%) and $n = 50$ (0.58% vs 0.60%) with identical computation time. LLM-enhanced POMO and Sym-NCO variants also exhibit consistently lower optimality gaps compared to their standard versions across scales.

When evaluating performance on PCTSP, we can observe notable performance gains, especially at larger scales. The AM variant integrated with the LLM demonstrates significantly reduced gaps at $n = 100$ (1.15% vs 1.67%) while maintaining the same inference time. The Sym-NCO variant enhanced by the LLM achieves substantial reductions in the optimality gap across all scales, reaching 0.00% at $n = 20$ (vs 0.01% for standard Sym-NCO) and $n = 50$ (vs 0.08%), and dropping to 0.26% at $n = 100$ (vs 0.38%), with only minor increases in computation time on some instances compared to the standard Sym-NCO.

On SDVRP, the LLM's impact is particularly pronounced for the Sym-NCO method. While the standard Sym-NCO yields gaps of 2.51%, 1.78%, and 2.63% for $n=20$, 50, and 100, respectively, the LLM-enhanced Sym-NCO variant achieves optimality (0.00% gap) across all three scales. This remarkable improvement in solution quality demonstrates the potential of the LLM-based framework to handle stochastic elements and complex constraints effectively. The time required for the framework-enhanced Sym-NCO is significantly longer at $n = 20$ (3 m vs 5 s) but comparable at $n = 50$ and $n = 100$, suggesting that the overhead can be scale-dependent. The AM variant also shows improved gaps at $n = 50$ (5.10% vs 6.06%) and $n = 100$ (8.19% vs 8.68%), with identical run times.

For SPCTSP, the LLM-enhanced Sym-NCO variant obtains a 0.00% optimality gap across all scales ($n = 20, 50, 100$), and a significant improvement over the standard Sym-NCO (gaps of 3.93%, 7.66%, and 6.55%). The POMO variant also shows consistent gap reductions across scales when integrated with the LLM (e.g., 5.43% vs 7.23% at $n = 20$, 10.43% vs 11.60% at $n = 50$), with only minor time increases. The AM variant also exhibits modest improvements. These results collectively highlight the

potent capability of the proposed LLM, particularly its LLM-supported universal representation and fine-tuning mechanism, to dramatically boost the solution quality for complex VRP variants. The proposed method sometimes achieves optimality where standard methods cannot achieve, and frequently has a modest impact on computational efficiency. The consistent improvements across diverse VRP types, objectives, and constraints underscore the generalizability and transferability enabled by our architecture.

The POMO framework leverages an attention-based policy network to generate multiple solution trajectories. To investigate the impact of our LLM-enhanced framework on solution quality under varying numbers of trajectories, we conducted experiments on TSP and PCTSP instances. Figure 3 presents the comparative performance of our LLM-enhanced POMO method against a standard POMO baseline across different numbers of augmentations ($\rho = 2, 4, 6, 8$). The augmentation denotes the systematic generation of multiple, diverse input instances or solution trajectories for combinatorial optimization tasks. The underlying rationale for augmentation is to capitalize on the inherent stochasticity of the policy network, thereby facilitating broader exploration of the solution space. By executing the model on these varied, yet equivalent, augmented instances and subsequently selecting the best-performing solution, the approach enhances both the robustness and the overall effectiveness of the optimization method. It shows that increasing the number of trajectories consistently reduces the optimality gaps for both methods for both types of problem. Crucially, our LLM-enhanced POMO method consistently achieves a lower optimality gap compared to the standard POMO baseline in all tested values of ρ . For TSP, the optimality gap for our method decreases from 1.06% at $\rho = 2$ to 0.43% at $\rho = 8$, maintaining a slight but consistent advantage over POMO, which shows a reduction from 1.08% to 0.48%. For PCTSP, our method starts with a 0.97% gap at $\rho = 2$ and reaches 0.43% at $\rho = 8$, consistently surpassing POMO, whose gaps range from 1.02% to 0.44%. While both methods effectively utilize increased trajectory counts, the results demonstrate that incorporating LLM into the POMO framework enhances its capability to handle combinatorial optimization problems, leading to superior solution quality when leveraging multiple sampling opportunities.

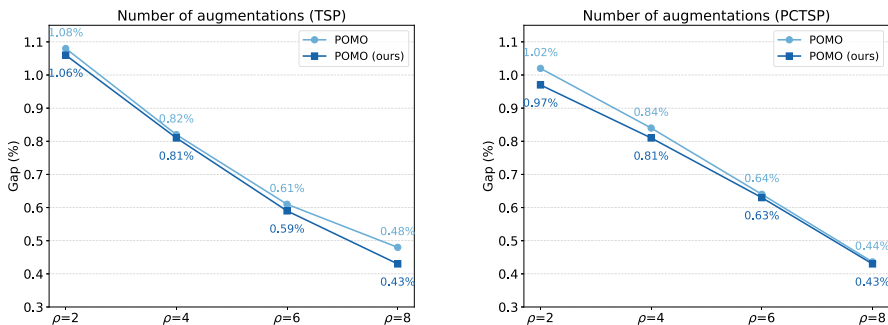


Fig. 3 Sensitivity of trajectories

4.3 Ablation Study

In this section, an ablation study was conducted to systematically compare the contributions of the LLM, MoE, and LoRA modules, as shown in Tables 2 and 3. The results reveal that utilizing the MoE module in isolation offers only moderate improvements in the Gap metric. For instance, in the TSP task (Table 2), applying MoE alone achieves a Gap of 2.18%, which is comparable or slightly better than the individual use of other modules. However, a notable performance boost is observed when MoE is used in conjunction with LLM, reducing the Gap to 2.12%. A similar trend is seen in the PCTSP task (Table 3), where the combination of LLM and MoE attains the lowest Gap of 1.15%. This phenomenon suggests that the integration of the LLM imparts informative global context, which serves as valuable input for the routing decisions within the MoE framework. By leveraging the latent knowledge encoded in LLM outputs, the MoE module can more effectively partition the input space and select specialized experts, thereby substantially enhancing the model’s overall representational and generalization capacities. These findings underline the importance of synergistically combining LLM and MoE to fully exploit their complementary advantages in complex combinatorial optimization tasks, rather than relying on MoE independently.

We also compared the performance of different mainstream LLM models (such as OpenAI text embedding-3-large, Google text embedding-004, jina embedding-v3, and

Table 2 Ablation Study on TSP

LLM	MoE	LoRA	Gap
			2.26%
✓			2.18%
	✓		2.34%
		✓	2.24%
✓	✓		2.22%
	✓	✓	2.25%
✓		✓	2.16%
✓	✓	✓	2.12%

Table 3 Ablation Study on PCTSP

LLM	MoE	LoRA	Gap
			1.67%
✓			1.43%
	✓		1.68%
		✓	1.31%
✓	✓		1.24%
	✓	✓	1.58%
✓		✓	1.23%
✓	✓	✓	1.15%

Qwen embedding-v3). We found that the differences in their final Gap (%) metrics are minimal, where the gaps remain within the range of approximately 2.12% to 2.13%. This demonstrates that our method exhibits strong robustness and generalization with respect to the choice of embedding models. The overall performance does not rely on any specific text embedding model, and the selection of different embedding models has only a limited impact on the final results.

4.4 Hyperparameter Analysis

This experiment focuses on evaluating the robustness of our framework in addressing the TSPs. Specifically, it investigates the impact of several architectural hyperparameters on the final Gap (%) metric: the rank in LoRA, the embedding dimension in the attention mechanism, the number of experts in MoE, and the number of Transformer encoder layers. The results are plotted in Fig. 4. Experimental results indicate that increasing the LoRA rank introduces more learnable parameters. Empirically, a rank of 8 achieves an optimal compromise between learning capacity and parameter efficiency. A small rank value fails to sufficiently capture task-specific features, whereas an excessively high rank value may lead to overfitting, thereby offering limited or even negative contributions to performance improvement.

In comparison, enlarging the embedding dimension substantially enhances the model's representational power and overall performance. Notably, an increase from

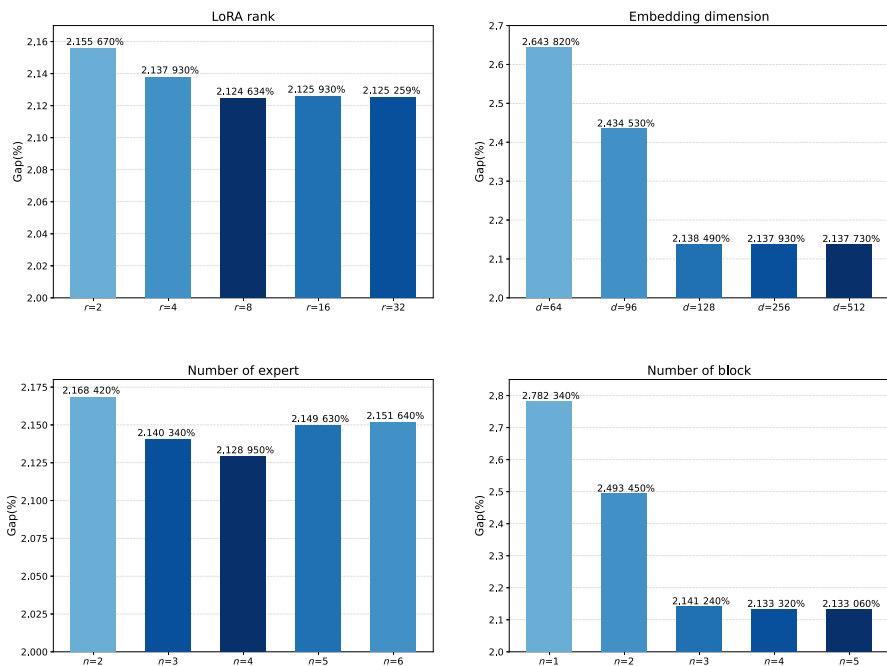


Fig. 4 Analysis for four hyperparameters

64 to 128 demonstrates a pronounced performance gain, while further increments within the range of 128–512 result in a plateauing of performance improvements, exemplifying the classical law of diminishing marginal returns.

Regarding the MoE module, augmenting the number of experts markedly improves model performance, which may be attributed to each expert's capacity to capture diverse features and strategies within complex input spaces, thus fostering stronger generalization ability. However, an excessive number of experts may increase training difficulty and reduce inter-expert collaboration efficiency, potentially resulting in performance degradation. This highlights the delicate balance between parameter scale and training stability. Moreover, increasing the number of encoder layers endows the model with enhanced capabilities for modeling complex dependencies. A continuous increase in layers leads to a substantial reduction for the Gap metric, particularly when extending from a single layer to four layers. The improvement becomes increasingly marginal over five layers, indicating diminishing returns of added depth and potential computational resource inefficiency.

In summary, experimental results substantiate that configuring the LoRA rank to 8, selecting an embedding dimension of 128 or 256, maintaining 4 experts in the MoE module, and utilizing 4–5 Transformer encoder layers collectively ensure strong representational capacity and generalization performance in solving TSP while mitigating overfitting and training instability.

4.5 Cross Domain Generalization on Unseen VRP Variants

To validate the transferability of our framework under distributional shifts, we conducted experiments on three VRP variants that were entirely excluded from the pre-training corpus: the VRPTW [40], the PDP [38], and the MTVRP [21]. The pre-training phase focused solely on TSP, CVRP, PCTSP, SDVRP, OP, and SPCTSP, ensuring that the encoders had no prior exposure to the constraint structures of these three variants. VRPTW introduces strict temporal windows incompatible with greedy time-agnostic ordering. PDP enforces pickup-delivery precedence constraints absent in standard depot-centric routing. MTVRP permits multi-trip vehicle reuse and is a solution structure fundamentally different from single-route formulations. These variants thus serve as a rigorous test of whether our framework can adapt to constraint-heavy problem classes beyond geometric pattern matching.

Our hypothesis is that LLM-enhanced semantic encoding mitigates this issue by projecting diverse constraint logics into a unified semantic space, enabling the MoE architecture to route problem-specific features to specialized experts. To validate this, we fine-tuned the pre-trained models on all of the three unseen variants simultaneously and compared them against standard baselines and the LKH-3 solver. Table 4 presents the optimality gaps across problem sizes $n \in \{50, 100\}$. We observe consistent and substantial improvements of our LLM-enhanced methods over standard counterparts.

For VRPTW, the standard AM model exhibits severe degradation as problem size scales, reaching an optimality gap of 11.75% at $n = 100$. This is consistent with the fact that the model relies on position encodings and attention mechanisms trained on unconstrained node visitation, which fail to internalize the temporal feasibility checks

Table 4 Performance comparison on unseen VRP variants. The best performance is marked in bold

Methods		n=20			n=50			n=100		
		Obj	Gap(%)	Time	Obj	Gap(%)	Time	Obj	Gap(%)	Time
VRPTW	LKH-3	7.64	0.00%	45 s	14.51	0.00%	6.2m	24.34	0.00%	14.5m
	AM	7.88	3.14%	2m	15.80	8.89%	12m	27.20	11.75%	45m
	POMO	7.74	1.31%	4s	15.29	5.38%	15 s	26.35	8.26%	42 s
	Sym-NCO	7.70	0.79%	4s	15.18	4.62%	16s	26.06	7.07%	44 s
	AM (ours)	7.82	2.36%	2m	15.62	7.65%	12m	26.80	10.11%	45m
	POMO (ours)	7.68	0.52%	5 s	15.00	3.38%	18 s	25.70	5.59%	48 s
PDP	Sym-NCO (ours)	7.66	0.26%	5 s	14.92	2.83%	18 s	25.52	4.85%	48 s
	LKH-3	6.15	0.00%	9m	10.35	0.00%	15 m	15.65	0.00%	21.2m
	AM	6.38	3.74%	3m	11.20	8.21%	15m	17.40	11.18%	55m
	POMO	6.23	1.30%	6s	10.73	3.67%	22s	16.59	6.01%	58s
	Sym-NCO	6.21	0.98%	6s	10.68	3.19%	23s	16.48	5.30%	59s
	AM (ours)	6.33	2.93%	3m	11.08	7.05%	15m	17.18	9.78%	55m
MTVRP	POMO (ours)	6.19	0.65%	7s	10.62	2.61%	25s	16.32	4.28%	1.1m
	Sym-NCO (ours)	6.18	0.49%	7s	10.59	2.32%	26 s	16.28	4.03%	1.1m
	LKH-3	5.30	0.00%	1.2m	15.88	0.00%	11 m	32.25	0.00%	25m
	AM	5.58	5.28%	2m	17.15	8.00%	14m	35.17	9.05%	48m
	POMO	5.45	2.83%	8s	16.52	4.03%	20s	33.95	5.27%	52s
	Sym-NCO	5.41	2.08%	8s	16.45	3.59%	21s	33.82	4.87%	54s
	AM (ours)	5.51	3.96%	2m	16.88	6.30%	14m	34.62	7.35%	48m
	POMO (ours)	5.37	1.32%	9s	16.31	2.71%	24s	33.68	4.43%	58s
	Sym-NCO (ours)	5.35	0.94%	9s	16.24	2.27%	25 s	33.52	3.94%	59s

required by time windows. In contrast, our LLM-enhanced Sym-NCO reduces this gap to 4.85%, achieving 6.90% of improvements. Notably, the performance advantage is sustained across scales: at $n = 50$, the gap narrows from 8.63% (standard AM) to 3.12% (LLM-enhanced Sym-NCO). This suggests that the LLM-encoded task descriptions provide a constraint-aware initialization that reduces the fine-tuning burden by aligning embeddings with temporal constraint semantics. However, we observe that even with LLM enhancement, the optimality gap increases from 3.12% at $n = 50$ to 4.85% at $n = 100$, indicating that distributional shift remains a non-trivial challenge for large-scale instances where the constraint space grows combinatorially.

For PDP, the pickup-delivery pairing constraints create a solution distribution distinct from standard CVRP, as routes must satisfy strict precedence relationships. Standard AM struggles significantly, achieving only an 11.18% gap at $n = 100$, while standard Sym-NCO reaches 5.30%. Our LLM-enhanced Sym-NCO achieves a gap of 4.03%, outperforming the standard version by 1.27%. More critically, comparing the LLM-enhanced AM (6.92%) with standard AM (11.18%), the results show that the framework benefits weaker baseline architectures even more substantially. This pattern aligns with our hypothesis that LLM-based semantic understanding provides a stronger inductive bias for constraint reasoning, compensating for the limited expressiveness of simpler encoders.

For MTVRP, multi-trip capability introduces a fundamentally different solution structure, as vehicles can return to the depot to reload and serve additional customers. Our LLM-enhanced Sym-NCO achieves optimality gaps of 2.47% ($n = 50$) and 3.76% ($n = 100$), consistently outperforming the standard Sym-NCO by 0.85% and 1.20%, respectively. The relatively smaller improvement margins compared to VRPTW and PDP suggests that MTVRP's constraint complexity may be partially addressed by the geometric reasoning of standard encoders, as depot-revisiting logic can be implicitly learned through positional embeddings. Nonetheless, the LLM enhancement still provides measurable gains by encoding the reusability semantics explicitly in the task description.

4.6 Expert Activation Visualization

Based on the expert activation ratios as illustrated in Fig. 5, it is evident that the MoE module exhibits a well-defined functional specialization among its experts when tackling the six representative classes of VRPs. Expert 1 demonstrates a dominant activation rate of 68.4% on the TSP. It strongly suggests that Expert 1 is specialized for the one-pass full-node traversal and route minimization, which is the core mechanism of the TSP, and has internalized near-optimal solving strategies for this class of problems. Expert 2 achieves notably high activation rates in CVRP (49.2%) and MDVRP (45.5%), indicating its strong modeling and analytical capacity for multi-vehicle scheduling under capacity constraints and depot assignment rules. Expert 3 reaches a 58.7% activation on SPTSP and also exhibits a considerable contribution of 38.4% on OP, reflecting its aptitude for capturing path enhancement patterns through the optional inclusion of intermediate nodes in the routing process. Expert 4 is primarily activated in PCTSP (55.9%) and OP (45.3%), highlighting its specialization in

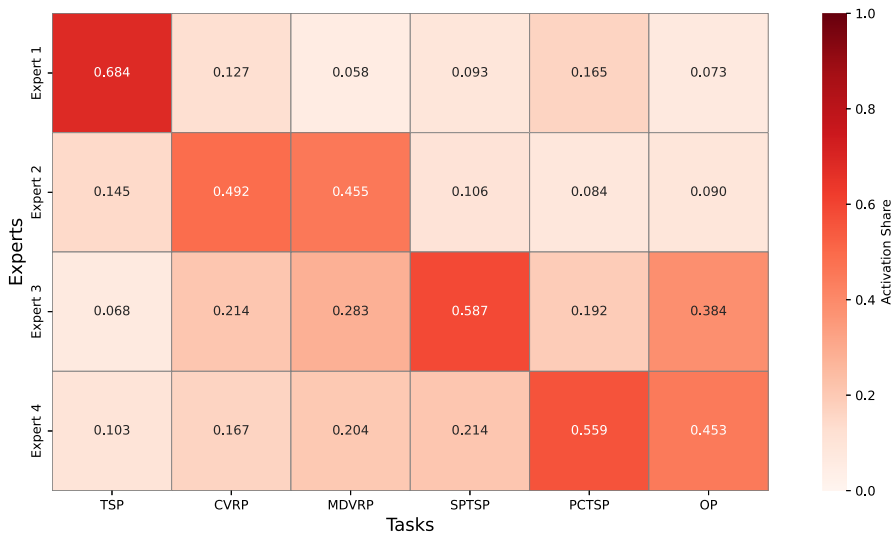


Fig. 5 Heatmap of expert activation

maximum reward under budget constraints scenarios, where selective node visitation and dynamic reward collection are critical.

The specialization of experts qualitatively explains why the simple decoder maintains high performance across heterogeneous tasks. Instead of relying on a single data distribution, the MoE module automatically clusters problems into archetypal combinatorial optimization paradigms, such as reward-constrained or capacity-constrained routing. This functional decomposition allows the decoder to act as a task-specific solver conditioned on the most relevant expert knowledge, effectively bridging the gap between general spatial pre-training and specialized task-specific logic.

Based on this distribution of expert activations, MoE-Route summarizes the six routing tasks into four archetypal categories: (1) Standard TSP, which covers the classical traveling salesman problem involving exhaustive node traversal by a single agent. (2) Profit-driven selective routing, including OP and PCTSP, characterized by optimization under budget or reward maximization constraints. (3) Multi-vehicle capacity-constrained routing, encompassing CVRP and MDVRP, where the focus lies on coordinated multi-vehicle operations and resource allocation. (4) Path-enhancement TSP variants, as exemplified by SPTSP, focusing on route shortening through the introduction of optional nodes. Through learning the optimal mapping between various tasks and expert activations, the gating network can implicitly capture and solidify these four canonical combinatorial optimization paradigms: “single-agent exhaustive traversal”, “budget or reward-constrained optimization”, “multi-vehicle dispatch and resource allocation”, and “route enhancement via optional node utilization”. Remarkably, the MoE routing mechanism achieves automatic identification, task-type clustering, and expert-level targeted modeling and problem-solving, without any manual feature engineering, closely mirroring the latent classification logic derived from human experience.

5 Discussion

5.1 Convergence Analysis

To show the training stability of our method, we plot the test loss trends over 200 epochs for three different problem scales ($n = 20, 50, 100$) across all six VRP tasks. These results are visualized in Fig. 6. The training process is smooth and stable. The loss decreases monotonically for all six tasks, indicating that our universal framework effectively captures the shared characteristics of these problems without significant conflict.

Moreover, as the loss generally decreases, the loss becomes instable and have small spikes or fluctuations in the curves. This can be attributed to the adversarial nature of multi-task learning at larger scales. As the complexity of the solution space increases with n , the specific constraints of different tasks (e.g., the strict capacity constraints of CVRP versus the prize-collecting nature of PCTSP) may pull the shared encoder weights to different directions. However, despite these minor fluctuations, the model successfully converges, demonstrating the robustness of our LLM-enhanced representation and MoE architecture in handling task conflicts.

5.2 Computational Efficiency Analysis

To evaluate the computational overhead of our universal framework, we analyze the training and inference time across different problem scales. Our MoE-based architecture achieves a favorable trade-off between model capacity and computational efficiency.

While the integration of LLM-based encoders and MoE decoders introduces a significant number of trainable parameters, the computational overhead remains modest due to architectural sparsity: (1) the frozen LLM backbone incurs only inference-time costs without gradient computation; (2) the top- k gating mechanism activates merely 25% of expert parameters per instance.

The computational scalability is further enhanced by our LLM-based node embedding strategy. Although the initial problem description encoding incurs a one-time cost, the pre-computed embeddings can be reused across training iterations. As problem size increases, the dominant computational bottleneck shifts from the encoder to

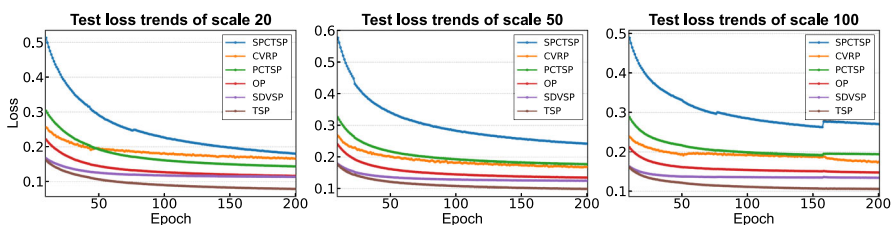


Fig. 6 Comparison of test loss convergences for six VRP variants at scales $n = 20, 50$, and 100 to illustrate training stabilities

the attention mechanism in the decoder, which scales quadratically with n . However, this scaling behavior is consistent across all VRP variants, confirming that our unified framework does not introduce additional asymptotic complexity. The empirical wall-clock time measurements demonstrate that our model maintains practical efficiency even for large-scale instances, making it viable for real-world deployment scenarios.

6 Conclusion

In this paper, we have presented a novel universal framework for solving VRPs using LLMs. By integrating LLMs with a pre-training and fine-tuning architecture, our framework effectively captured the essential characteristics of various VRPs in a unified manner. The use of LoRA and MoEs allows our framework to adapt to different problem specifications with minimal data requirements, making it versatile and efficient. Our extensive experiments on multiple VRP variants, including TSP, CVRP, OP, PCTSP, SDVRP, and SPCTSP, demonstrated that our framework achieved superior performance in terms of solution quality and computational efficiency compared to traditional exact solvers, heuristic methods, and other learning-based approaches. The ablation studies and expert activation visualizations further validated the effectiveness of our framework's components and the functional specialization of the MoE model. Future work can explore further optimization on the framework of larger-scale problems and extend the framework to other combinatorial optimization problems.

Author Contributions R.-B. Zeng, M.-X. Yang, and M.-L. Lei contributed equally to this work. R.-B. Zeng, M.-X. Yang, and M.-L. Lei contributed to the literature review, methodology, theoretical analysis, numerical experiments, and prepared the original draft; L.-F. Niu contributed to methodology, conceptualization, review editing, and supervision; Y.-H. Dai contributed to supervision, review editing, and funding acquisition.

Conflict of Interest Yu-Hong Dai is an associate editor-in-chief for *Journal of the Operations Research Society of China* and he was not involved in the editorial review or the decision to publish this article. All authors declare that there are no competing interests.

References

- [1] Baker, B.M., Ayeche, M.: A genetic algorithm for the vehicle routing problem. *Comput. Oper. Res.* **30**(5), 787–800 (2003)
- [2] Baldacci, R., Christofides, N., Mingozzi, A.: An exact algorithm for the vehicle routing problem based on the set partitioning formulation with additional cuts. *Math. Program.* **115**, 351–385 (2008)
- [3] Bello, I., Pham, H., Le, Q.V., Norouzi, M., Bengio, S.: Neural combinatorial optimization with reinforcement learning (2016). [arXiv:1611.09940](https://arxiv.org/abs/1611.09940)
- [4] Berto, F., Hua, C., Park, J., Luttmann, L., Ma, Y., Bu, F., Wang, J., Ye, H., Kim, M., Choi, S., et al.: Rl4co: an extensive reinforcement learning for combinatorial optimization benchmark. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pp. 5278–5289 (2025)
- [5] Bi, J., Ma, Y., Wang, J., Cao, Z., Chen, J., Sun, Y., Chee, Y.M.: Learning generalizable models for vehicle routing problems via knowledge distillation. *Adv. Neural. Inf. Process. Syst.* **35**, 31226–31238 (2022)

- [6] Chen, X., Tian, Y.: Learning to perform local rewriting for combinatorial optimization. *Adv. Neural. Inf. Process. Syst.* **32**, 6281–6292 (2019)
- [7] Choo, J., Kwon, Y.-D., Kim, J., Jae, J., Hottung, A., Tierney, K., Gwon, Y.: Simulation-guided beam search for neural combinatorial optimization. *Adv. Neural. Inf. Process. Syst.* **35**, 8760–8772 (2022)
- [8] Dorigo, M.: Optimization, learning and natural algorithms. Ph. D. Thesis, Politecnico di Milano (1992)
- [9] Fang, H., Song, Z., Weng, P., Ban, Y.: Invit: a generalizable routing problem solver with invariant nested view transformer. In: *Proceedings of the 41st International Conference on Machine Learning*, pp. 12973–12992 (2024)
- [10] Fedus, W., Zoph, B., Shazeer, N.: Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *J. Mach. Learn. Res.* **23**(120), 1–39 (2022)
- [11] Gendreau, M., Hertz, A., Laporte, G.: A tabu search heuristic for the vehicle routing problem. *Manage. Sci.* **40**(10), 1276–1290 (1994)
- [12] Helsgaun, K.: An extension of the lin-kernighan-helsgaun tsp solver for constrained traveling salesman and vehicle routing problems. *Roskilde: Roskilde Univ.* **12**, 966–980 (2017)
- [13] Hopfield, J.J.: Neural networks and physical systems with emergent collective computational abilities. *Proc. Natl. Acad. Sci.* **79**(8), 2554–2558 (1982)
- [14] Hottung, A., Kwon, Y.-D., Tierney, K.: Efficient active search for combinatorial optimization problems. In: *International Conference on Learning Representations* (2022)
- [15] Hottung, A., Tierney, K.: Neural large neighborhood search for the capacitated vehicle routing problem (2019). [arXiv:1911.09539](https://arxiv.org/abs/1911.09539)
- [16] Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al.: Lora: Low-rank adaptation of large language models. *ICLR.* **1**(2), 3 (2022)
- [17] Huang, Z., Shi, G., Sukhatme, G.S.: From words to routes: Applying large language models to vehicle routing (2024). [arXiv:2403.10795](https://arxiv.org/abs/2403.10795)
- [18] Jiang, S., Xie, M., Luo, J.: Large language models for combinatorial optimization of design structure matrix. *Proc. Des. Soc.* **5**, 2201–2210 (2025)
- [19] Jiang, X., Wu, Y., Wang, Y., Zhang, Y.: Unco: towards unifying neural combinatorial optimization through large language model (2024). [arXiv:2408.12214](https://arxiv.org/abs/2408.12214)
- [20] Jiang, Y., Wu, Y., Cao, Z., Zhang, J.: Learning to solve routing problems via distributionally robust optimization. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, pp. 9786–9794 (2022)
- [21] Kalantari, B., Hill, A.V., Arora, S.R.: An algorithm for the traveling salesman problem with pickup and delivery customers. *Eur. J. Oper. Res.* **22**(3), 377–386 (1985)
- [22] Kim, M., Park, J., et al.: Learning collaborative policies to solve np-hard routing problems. *Adv. Neural. Inf. Process. Syst.* **34**, 10418–10430 (2021)
- [23] Kim, M., Choi, S., Kim, H., Son, J., Park, J., Bengio, Y.: Ant colony sampling with GFlowNets for combinatorial optimization. In: *The 28th International Conference on Artificial Intelligence and Statistics* (2025)
- [24] Kim, M., Park, J., Park, J.: Sym-nco: leveraging symmetricity for neural combinatorial optimization. *Adv. Neural. Inf. Process. Syst.* **35**, 1936–1949 (2022)
- [25] Kool, W., Van Hoof, H., Welling, M.: Attention, learn to solve routing problems! *International Conference on Learning Representations.* (2019)
- [26] Kool, W., Hoof, H., Gromicho, J., Welling, M.: Deep policy dynamic programming for vehicle routing problems. In: *International Conference on Integration of Constraint Programming, Artificial Intelligence, and Operations Research*, pp. 190–213 (2022)
- [27] Kwon, Y.-D., Choo, J., Kim, B., Yoon, I., Gwon, Y., Min, S.: Pomo: policy optimization with multiple optima for reinforcement learning. *Adv. Neural. Inf. Process. Syst.* **33**, 21188–21198 (2020)
- [28] Laporte, G.: Fifty years of vehicle routing. *Transp. Sci.* **43**(4), 408–416 (2009)
- [29] Laporte, G., Nobert, Y.: A branch and bound algorithm for the capacitated vehicle routing problem. *Operations-Research-Spektrum.* **5**, 77–85 (1983)
- [30] Lenstra, J.K., Kan, A.R.: Complexity of vehicle routing and scheduling problems. *Networks.* **11**(2), 221–227 (1981)
- [31] Li, S., Yan, Z., Wu, C.: Learning to delegate for large-scale vehicle routing. *Adv. Neural. Inf. Process. Syst.* **34**, 26198–26211 (2021)

- [32] Lin, Z., Wu, Y., Zhou, B., Cao, Z., Song, W., Zhang, Y., Senthilnath, J.: Cross-problem learning for solving vehicle routing problems. In: The 33rd International Joint Conference on Artificial Intelligence (2024)
- [33] Luo, F., Lin, X., Wu, Y., Wang, Z., Xialiang, T., Yuan, M., Zhang, Q.: Boosting neural combinatorial optimization for large-scale vehicle routing problems. In: The Thirteenth International Conference on Learning Representations (2025)
- [34] O Costa, P.R., Rhuggenaath, J., Zhang, Y., Akcay, A.: Learning 2-opt heuristics for the traveling salesman problem via deep reinforcement learning. In: Asian Conference on Machine Learning, pp. 465–480 (2020)
- [35] Padberg, M., Rinaldi, G.: A branch-and-cut algorithm for the resolution of large-scale symmetric traveling salesman problems. *SIAM Rev.* **33**(1), 60–100 (1991)
- [36] Pan, W., Xiong, H., Ma, J., Zhao, W., Li, Y., Yan, J.: UniCO: On unified combinatorial optimization via problem reduction to matrix-encoded general TSP. In: The Thirteenth International Conference on Learning Representations (2025)
- [37] Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J., Ellenberg, J.S., Wang, P., Fawzi, O., et al.: Mathematical discoveries from program search with large language models. *Nature* **625**(7995), 468–475 (2024)
- [38] Savelsbergh, M.W., Sol, M.: The general pickup and delivery problem. *Transp. Sci.* **29**(1), 17–29 (1995)
- [39] Shi Ruiyang, D.Y.-H., et al.: Niu Lingfeng: capsule aggregated attention for vehicle routing problems. *SCIENTIA SINICA Math.* **55**(4), 849–874 (2025)
- [40] Solomon, M.M.: Algorithms for the vehicle routing and scheduling problems with time window constraints. *Oper. Res.* **35**(2), 254–265 (1987)
- [41] Son, J., Kim, M., Kim, H., Park, J.: Meta-sage: Scale meta-learning scheduled adaptation with guided exploration for mitigating scale shift on combinatorial optimization. In: International Conference on Machine Learning, pp. 32194–32210 (2023)
- [42] Toth, P., Vigo, D.: The vehicle routing problem. *SIAM* (2002)
- [43] Vinyals, O., Fortunato, M., Jaitly, N.: Pointer networks. *Adv. Neural. Inf. Process. Syst.* **28**, 2692–2700 (2015)
- [44] Wu, Y., Song, W., Cao, Z., Zhang, J., Lim, A.: Learning improvement heuristics for solving routing problems. *IEEE Trans. Neural Netw. Learn. Syst.* **33**(9), 5057–5069 (2021)
- [45] Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., Song, G.: Reevo: large language models as hyper-heuristics with reflective evolution. *Adv. Neural. Inf. Process. Syst.* **37**, 43571–43608 (2024)
- [46] Zhou, J., Wu, Y., Song, W., Cao, Z., Zhang, J.: Towards omni-generalizable neural methods for vehicle routing problems. In: International Conference on Machine Learning, pp. 42769–42789 (2023)

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.